CSE 144 Final Project Report
Transfer Learning Challenge

Andrew Nguyen (CruzID: anguy522)
Aditya Davanam (CruzID: adavanam)
Franklin Wang (CruzID: frywang)
Spring 2026

Introduction

Problem goal and setting.

The goal of the transfer learning challenge was to apply transfer learning techniques covered in class to take a pre-trained image classifier and train the model to learn a new dataset with 100 classes with a low amount of training data(only 10 images per class).

Why transfer learning is appropriate.

Transfer learning is appropriate here because the dataset here is too small to train a deep learning model from scratch. Doing so would lead to severe overfitting in all layers of the model. By doing transfer learning with a stronger pre-trained model, we can preserve the existing feature extraction capabilities of previously trained models and only train a classifier head from scratch.

Brief of your approach and main result.

Our approach focused on experimenting with different model architectures like EfficientNet, ConvNext, Vision Transformers, and SWIN transformers. After finding the best performing architectures, we would experiment with hyperparameters to push our validation accuracy as far as possible.

Dataset

Number of classes and dataset sizes (train/val/test)

The data consisted of 100 classes to learn and we split the data into 80% training data and 20% validation data. We used the provided test data to test our model via kaggle submissions.

Directory structure and label mapping details

The names of the classes were integers, rather than longer strings describing the actual class. These integers corresponded to the actual class index associated with those images. However, the datasets.ImageFolder function maps indices to classes alphabetically, so there were mixmatches between class and index. To solve this, we made a custom sorting function to re-sort the internal class to index mappings that result from using the Subset function, which was used in an earlier cell in the notebook. We then passed this into a DataLoader and all subsequent code would work as expected.

Preprocessing and Data Augmentation used

As part of our preprocessing we made the images to have a resolution of 384x384 and normalized the RGB values to be that of the mean and standard deviation values of the ImageNet dataset.

Our data augmentation consisted of the following:

- Random crop between 60% and 100% of the original image
- Random horizontal flip with p=0.5
- Random Rotation of maximum 7 degrees
- Mild ColorJitter (brightness=0.2, contrast=0.2, saturation=0.2, hue=0.03)
- For our SWIN Transformer model, we used MixUp with alpha=0.2, but no mixup for EfficientNetV2.

Implementation

Model

Pretrained backbone Used And Why

We used EfficientNetV2-Small as it performed the best when experimenting with various CNN models using the same hyperparameters. We then additionally used SWIN-Base as we learned from the presentations that many other groups were finding success with SWIN transformer models

Architecture Changes

We immediately knew that for both models, we would have to replace the classifier head with a new one, as the old one was designed for learning an entirely different dataset. We replaced the classifier heads of both models with a one that had a single linear layer with dropout(p=0.4).

Through various experiments with freezing/unfreezing parts of the backbone, we found that it was most beneficial to keep the entire backbone unfrozen, but with different learning rates for different parts(elaborated on in “Fine-Tuning Strategy”).

We then found that the models were still overfitting slightly, so we added layer dropout with a small probability and neuron dropout with small probability(We only added neuron dropout for the SWIN model as there were no existing dropout modules to modify in EfficientNetV2-Small).

Fine-Tuning Strategy

We kept the backbone fully unfrozen with different learning rates for the head, the last 2 feature extractors, and rest of the feature extractors. We found this to be more beneficial than unfreezing layers as training went on.

Training

Loss function and Optimizer

We used Cross Entropy loss and PyTorch's built-in AdamW optimizer.

Learning Rate, Scheduler, Batch Size, Epochs, Weight Decay

We implemented different slightly different hyperparameters for the two models we ensemble and they are listed below:

Hyperparameters	EfficientNetV2-Small	SWIN-Base
Batch Size	32	32
Epochs(Training/Validation)	75	75
Epochs(Final Submission)	100	75
Classifier Neuron Dropout Rate	0.4	0.4
Backbone Layer Dropout Rate	0.15	0.2
Backbone Neuron Dropout Rate	N/A	0.15
Classifier Learning Rate	1e-3	1e-3
Learning Rate for Last 2 Feature Extractor Blocks	1e-4	1e-4
Learning Rate for Rest of Backbone	1e-5	2e-5
Scheduler	Cosine Annealing LR with Linear Warm Restarts	Cosine Annealing LR
Label Smoothing	0.05	0.05
AdamW Weight Decay	1e-3	1e-3
Mixup Alpha	No Mixup	0.2

Hardware/Software Environment

We paid for an online cloud compute via runpod.io. Our VM had an Nvidia A100SXM GPU with a 32-core CPU. We used Jupyter Notebook within the VM.

Experiments

Baseline Setup

For our baseline setup, we first used convNext-tiny and did very minor fine-tuning to hyperparameters to see what yielded decent results. We then took those hyperparameters and

applied them to various network architectures, picking the 2 best performers. We then did further experimentation and hyperparameter tuning with those.

Hyperparameter Tuning Plan and Validation Method.

The goal of hyperparameter tuning was to improve accuracy and fix our accuracy plateau. We tuned learning rate, scheduler, batch size, froze/unfroze layers, and varied the number of epochs we trained for. We used manual tuning and incremental changes to move the hyperparameters around. To validate our trained weights, we used an 80/20 train/val split and evaluation per epoch, saving the model weights of the best epoch.

Ablations

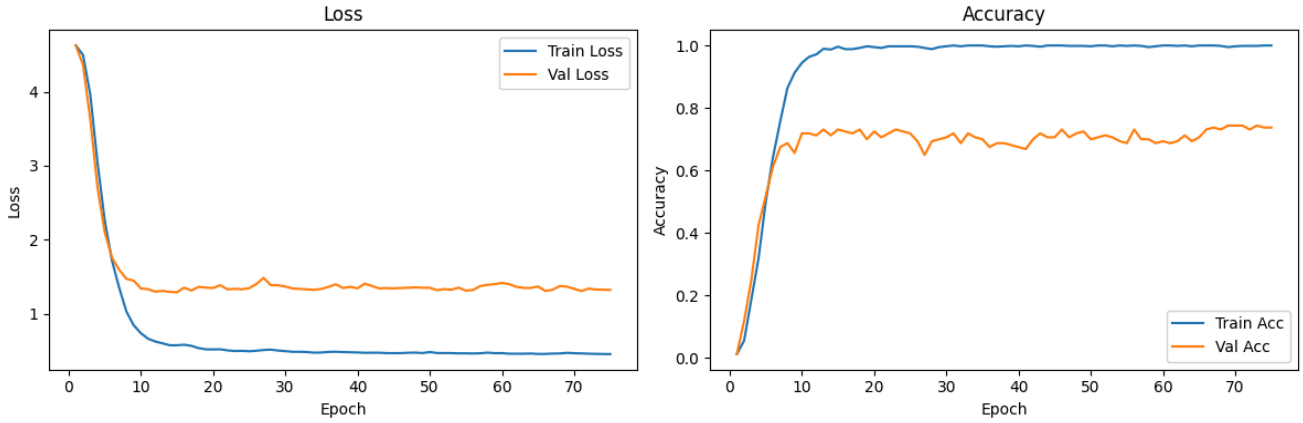
We used data augmentations like mild rotation, color jitter, and horizontal flipping to increase the variance within the dataset. For the model size, we first used smaller models like convNext-tiny and EfficientNetB0, but moved to larger ones like SWIN-Base and EfficientNetV2 once we decided to pay for a VM with higher specs. We arrived upon our freezing strategy by testing the model performance upon freezing different amounts of parameters. We then chose the best-performing setup that we were able to find within the time constraints of the project. We also experimented with different learning rate schedulers, finding very very minor differences between them, ultimately settling on Cosine Annealing for both models. We also varied the number of epochs to see where the trained model's accuracy on the validation data would plateau.

Results

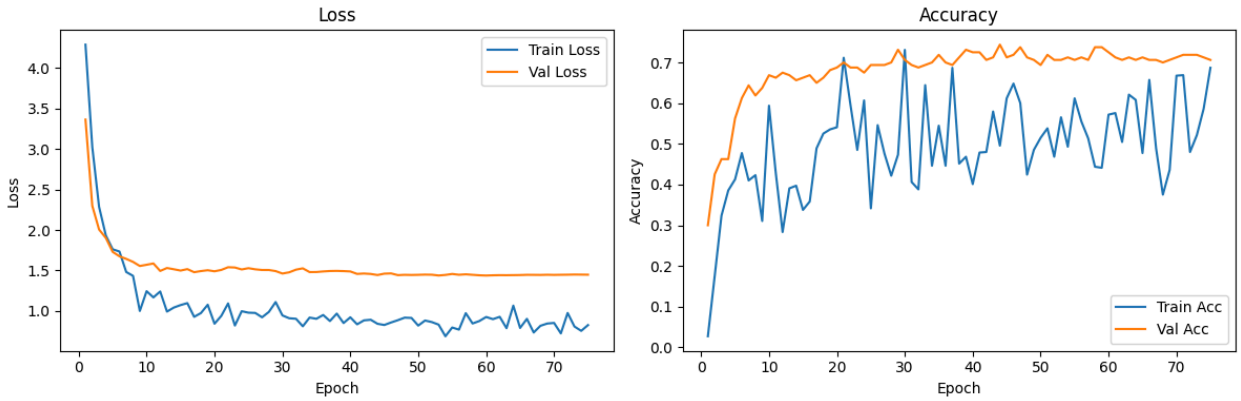
Training/Validation Accuracy (and loss) and Key Curves/Plots

Throughout training, we monitored both training and validation accuracy to evaluate model performance and detect overfitting. Our initial EfficientNet-B0 baseline achieved approximately 55% validation accuracy. After experimenting with different models, freezing strategies, learning rates, data augmentations, and learning rate scheduling, performance improved significantly. Our best configuration achieved approximately 74% validation accuracy after 75 epochs, with the highest score occurring around the 50-60 epoch range. Training and validation curves showed steady improvement throughout training, suggesting that the model continued to benefit from additional epochs and had not fully converged earlier in the training process.

EfficientNetV2-Small Train/Val Loss and Accuracy Curves:

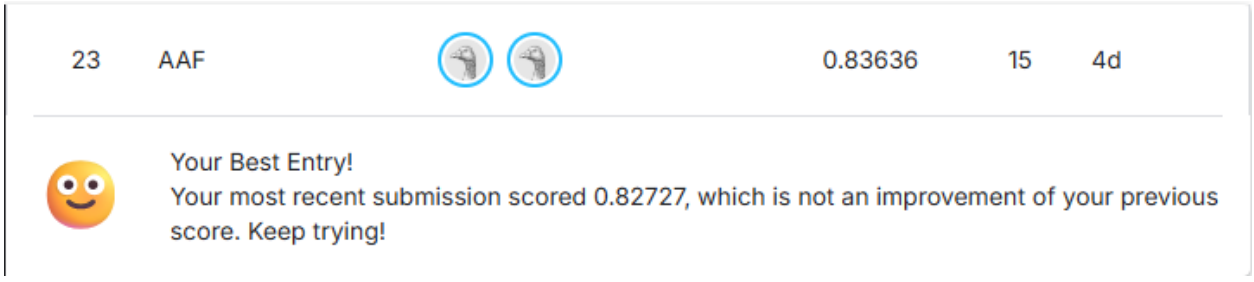


SWIN-Base Train/Val Loss and Accuracy Curves:



Kaggle Public Leaderboard Score (and submission settings)

The submission used an ensembling of predictions made by EfficientNetV2-Small and SWIN-Base. For our final submission, we train the model on the entire training dataset and submit it to Kaggle to improve accuracy on the final test set.



Discussion

What Worked Best And Why

The best performing setups used EfficentNetV2-Small and SWIN-Base, both reaching 73-74% validation accuracy. We used different learning rates for different parts of the model as we didn't want to change the high-level feature extractors too much, but have stronger learning for the feature extractors designed to extract small details. The learning rate of the classifier had

to be high as it was being trained from scratch. We used both layer-level dropout and neuron-level dropout to improve the generalization capabilities of the model. All other hyperparameters were tuned via manual experimentation.

Failure Cases, Overfitting/Underfitting Observations

It is clear from the train/val curve on EfficientNetV2-Small that the model is overfitting the data, but we still found the highest validation accuracy using minimal data augmentation and no mixup. It is hard to tell if SWIN-Base was overfitting, as we used mixup, making the training accuracy very noisy, but we can see that the training loss continues to decrease while the validation loss plateaus. This may indicate overfitting in our SWIN-Base model as well.

Limitations and Concrete Next Improvements

The limitation of our approach was that we chose to use older models as our pre-trained models. We found from the presentations that other groups were using more modern models like SIGLIP and DINO to find higher accuracies. The dataset had 4 broader classes, but within each class were fine-grained distinctions. It seems that newer models are better at doing both kinds of classification than the older models. We feel that to make significant improvements in our accuracy, we would have to experiment with newer image classification models.

Reproducibility

Random Seeds and Determinism Settings

We use a fixed seed of 42 and toggled certain flags in pytorch to prioritize speed and optimal GPU usage over strict reproducibility. This however did not affect our models ability to consistently hit validation accuracies of 73-74%.

Package Versions / Environment Setup Steps

- Python 3.12.1
- PyTorch 2.11.0 (confirmed by print(torch.__version__) output in the notebook)
- Hardware: Nvidia A100SXM GPU
- Install dependencies: Torch, Torchvision, Matplotlib, Tqdm, Numpy, Certifi, Pillow, Pandas, Scikit-Learn

Exact Commands to Train and Generate submission.csv

To train, you can simply run the cells in order for both SWINBase.ipynb and EfficientNet.ipynb. If you would like to use a train/val split, set USE_VALIDATION to true. There is a cell to generate a CSV using TTA predictions and one to generate a CSV with regular predictions.

Ensemble.csv is a separate notebook meant to ensemble the predictions of both models and generate a CSV based on the softmax outputs of the models on the same image.

Team Contributions

Aditya Davanam

- Freeze/Unfreezing Parameters, Test-Time Augmentation, Experimentation with different models and hyperparameters, $20 Personal Investment For Cloud Compute

Andrew Nguyen

- Data Augmentation + Training Loop, Visualization, Experimentation with different models and hyperparameters

Franklin Wang

- Train/Val Stratified Split, Implementing Mixup, Report, Presentation

References

TorchVision Documentation / Pretrained Model Sources

- PyTorch Documentation. torch.optim — AdamW, ReduceLROnPlateau. https://pytorch.org/docs/stable/optim.html
- TorchVision Documentation. EfficientNet — torchvision.models.EfficientNet_V2_S_Weights.DEFAULT https://docs.pytorch.org/vision/2.0/models/efficientnetv2.html
- TorchVision Documentation: SWIN-Base - torchvision.models.SWIN_B_Weights.DEFAULT https://docs.pytorch.org/vision/main/models/generated/torchvision.models.swin_b.html
- TorchVision Documentation. Models and pretrained weights overview. https://pytorch.org/vision/stable/models.html

Papers and Tutorials

- Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019. https://arxiv.org/abs/1905.11946
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016. https://arxiv.org/abs/1512.03385
- (used as the initial baseline model)
- Liu, Z., et al. (2022). A ConvNet for the 2020s (ConvNeXt). CVPR 2022. https://arxiv.org/abs/2201.03545
- Liu, Z., et al. (2021). Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. ICCV 2021. https://arxiv.org/abs/2103.14030
- Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR 2019. https://arxiv.org/abs/1711.05101